

ABRA v2 — Handoff (for a fresh chat)

Everything a new session needs to continue the ABRA v2 build without re-deriving context. Repo: `C:\Users\willj\Projects\Pokemon\ABRA` (site also mirrored to `app/index.html`). Last updated 2026-07-23.

What ABRA is

A model family for **Pokémon Champions VGC, Reg M-B, Bo1 closed-sheet ladder** (this label matters — 99% of our 5,200+ real replays are Bo1 closed sheets; Bo3 open sheets is a *different* game, scaffolded for later into a separate `data/games.bo3.jsonl` bucket). Framed as "poker theory → Pokémon." Whimsical single-file site at `web/index.html` (buildless, opens via `file://`).

The v2 pivot (already decided — don't relitigate)

We proved with real games that **predicting the winner from team sheets is near-impossible in this format — even player-Elo ties a coin (0.687 vs 0.693)**. So v2 stops predicting outcomes and **supports decisions** (the poker/Metamon/CICERO/sports-analytics recipe). Full reasoning: `docs/LITERATURE-v2.md` . Architecture + build order + acceptance bars: `docs/ALAKAZAM-v2-spec.md` . Living model ledger: `docs/MODELS.md` . External review that triggered this: `docs/EXTERNAL-REVIEW-2026-07-23.md` (+ pdf).

Validated findings (the honest ledger)

- **Damage engine = exact** against the Smogon damage calculator (100% within 5%, gated in CI). `engine/validate_damage.js` , `data/damage-validation.json` .
- **Pre-game win% = coin** (Bo1 closed). MEDICHAM's win% is *below* chance (inverted — greedy policy backs fast/offensive teams). `data/winrate-backtest.json` .
- **Behaviour-clone (belief) = modest**: top-1 36% / top-3 72% on held-out human moves. `data/policy-eval.json` .
- **PORY (mid-game value) = WORKS**: log-loss 0.567 vs coin 0.693, beats material heuristic, calibrated (ECE 1.6%), honest clustered-by-game CI [0.548,0.583]. `data/pory-eval.json` . Proves the pivot. **Now wired into KADABRA** as a per-turn "you're at X%" chip.
- **CHOMP-EV (bring quality) = NULL at the ceiling**: on 1,205 held-out human games CHOMP's bring ranking ties a coin (log-loss 0.6918 vs 0.6931), an Elo, and a usage prior; winners barely more CHOMP-aligned than losers (0.512, CI [0.493,0.535]). Robust to forfeits; selection audit shows the bias favors CHOMP, so the null is conservative. Damage math stays validated — it's the bring *selection signal* that's at ceiling. `data/chomp-ev.json` , `engine/chomp_ev.js` . Honest negative that blocks a DITTO-style .

Model roster (final, snappy names)

Built + validated this session:

- **GURU** — meta/matchup matrix from REAL outcomes, Wilson CIs. `engine/guru.py` → `data/guru.js` (on the site). 
- **XATU** — opponent belief (item/ability/moves) from replays. `engine/xatu.py` → `data/xatu.js` . 

- **PORY** — mid-game win-prob value net. `engine/pory.py` → `data/pory.js` . ✓ (the win)

Kept / roles:

- **MEDICHAM** = validated damage engine/simulator (foundation). **CHOMP** = bring/lead EV. **SLOWKING** = team-preview Nash. **DITTO** = team-builder (PSRO). **KADABRA** = mid-game play-by-play coach. **ALAKAZAM** = the boss (in-battle decision engine, built LAST). **JOLTEON** = demoted to a usage-prior. **HYPNO** = opponent-type/exploitability. **DUSK** = endgame exact solver. **MEW** = self-play data engine. **ROTOM** = swappable gimmick module (currently Mega). **TRUBBISH/GARBODOR** = GIGO mascots.

Build order — DO NEXT (inputs first, capstone last)

1. ✓ MEDICHAM damage. 2. ✓ GURU + XATU + **PORY**.
2. ✓ **CHOMP-EV proof** — ran the winnable test (`engine/chomp_ev.js` → `data/chomp-ev.json`). Result: honest **NULL** — CHOMP's bring ranking ties a coin/Elo/usage prior on held-out games; robust to forfeits + selection-audited. Damage stays validated; the bring *selection* is at the format ceiling. Test + CI: `tests/test-chomp-ev.js` .
3. ✓ Wired **PORY's live win%** into KADABRA (per-turn "you're at X%" chip; `poryWin()` in `web/index.html`).
4. ✓ **SLOWKING preview-Nash** — built on GURU's real matchup matrix (`engine/slowking_preview.py` → `data/slowking-eval.json`). Equilibrium mix 0.84/0.16; exploitability Nash≈0 vs uniform 0.109; meta near-transitive at the archetype level (greedy≈Nash) but a real 3-cycle exists. Test + CI: `tests/test-slowking.py` .
5. **NEXT: refine SLOWKING to playstyle/set level** (stall / Trick Room / perish-trap / setup archetypes + XATU belief over the opponent's six) to expose the non-transitive cycles greedy can't see; then DITTO/PSRO team-builder → **ALAKAZAM** (assemble PORY value + XATU belief + MEDICHAM damage with KL-anchored search). *Also*: a belief-aware bring value (XATU + SLOWKING lead stage-game + PORY leaf) is the credible path to a CHOMP bring edge — re-run `chomp_ev.js` after to measure the lift over the null.

Hard constraints (honor these)

- **Compute:** CPU only, no GPU. Keep models small/CPU-trainable. **Inference:** tiny in-browser JSON (like `live.js`), no build step. **Data:** real human replays first (Bo1 closed).
- **Rigor bar (MIT-chair review):** every probability ships a proper score (log-loss/Brier) + CI (cluster by game where states are correlated) + honest baseline, persisted to JSON. No capstone until inputs clear their bars.
- **Standing user rules:** copyable / one-click for anything on the user's end; NO opening/presenting files in chat (`present_files` is banned); no tiny text on the site; short snappy names; concise replies; no dumb emojis.

Data pipeline (already set up)

- **Cloud scraper:** GitHub Action `.github/workflows/ingest.yml` (hourly, machine-independent) → grows `data/games.ladder.jsonl` . Repo `github.com/willhoop/abra` (user must keep Actions enabled; the separate `tests` workflow was disabled to stop failure emails).

- **Local backup:** scheduled task every 3h pulls Bo1 (main) + Bo3 (separate `games.bo3.jsonl`) + runs `engine/refresh-site-data.py` (regenerates `live.js`, `archetypes.json`, `kad-replays.js`).
- ~2,600 public games/day ceiling; store grows ~18k/week. Models get properly testable as it accumulates.
- `data/games.ladder.raw-logs.jsonl` is **gitignored** (large, local-only) — CI-run scripts (`eval_policy.py`, `pory.py`) guard-skip if it's absent.

How to verify things

`node engine/validate_damage.js` (needs `npm i the Smogon damage calculator`), `python3 engine/guru.py|xatu.py|pory.py`, `python3 engine/eval_policy.py`, `node engine/backtest_winrate.js`. Syntax-check the site by extracting `<script>` blocks and `node --check`. Always `cp web/index.html app/index.html` after site edits.